

Example: Using a Constant in a Budget Calculator

Let's say your budget calculator applies a **fixed tax rate** to all income. Instead of hardcoding the number repeatedly, use a constant.

```
python
CopyEdit
# Constants
TAX_RATE = 0.10 # 10% income tax

# Function to calculate net income after tax
def calculate_net_income(gross_income):
    tax_amount = gross_income * TAX_RATE
    net_income = gross_income - tax_amount
    return net_income

# Example usage
gross_income = float(input("Enter your gross monthly income: $"))
net_income = calculate_net_income(gross_income)

print(f"After {TAX_RATE * 100:.0f}% tax, your net income is:
${net_income:.2f}")
```

Example: if-else for Budget Status Check

```
python
CopyEdit
# User inputs
budget = float(input("Enter your total monthly budget: $"))
expenses = float(input("Enter your total monthly expenses: $"))

# Decision structure
if expenses > budget:
    print("⚠ Warning: You are over budget!")
elif expenses == budget:
    print("✅ You're right on budget.")
else:
    savings = budget - expenses
```

```
    print(f"🎉 Good job! You're under budget. You saved  
    ${savings:.2f}.")
```

What's Happening:

- If **expenses exceed budget**, it warns the user.
 - If **expenses equal budget**, it notes that you're on track.
 - If **expenses are under budget**, it calculates and displays the savings.
-

Example: Repetition with **while** Loop for Multiple Expenses

```
python  
CopyEdit  
# Initialize variables  
expenses = []  
add_more = True  
  
# Collect multiple expenses using a while loop  
while add_more:  
    category = input("Enter expense category (e.g., Rent, Food): ")  
    amount = float(input(f"Enter amount for {category}: $"))  
    expenses.append((category, amount))  
  
    # Ask user if they want to continue  
    response = input("Add another expense? (yes/no):  
").strip().lower()  
    if response != 'yes':  
        add_more = False  
  
# Summarize total expenses  
total = sum(amount for _, amount in expenses)  
  
# Display summary
```

```
print("\n--- Monthly Expense Summary ---")
for category, amount in expenses:
    print(f"{category}: ${amount:.2f}")
print(f"Total Expenses: ${total:.2f}")
```

What the **while** loop does:

- Repeats as long as the user says "yes"
- Collects multiple categories and amounts
- Ends cleanly when the user says "no"

Example: Iterating with a **for** loop over a list of expenses

```
python
CopyEdit
# Example expense data (could be user-generated)
expenses = [
    ("Rent", 1200),
    ("Groceries", 300),
    ("Utilities", 150),
    ("Transportation", 100),
    ("Entertainment", 80)
]

# Initialize total
total_expenses = 0

# Use a for loop to display each expense and compute total
print("\n--- Monthly Expenses ---")
for category, amount in expenses:
    print(f"{category}: ${amount:.2f}")
    total_expenses += amount

# Show total
print(f"\nTotal Expenses: ${total_expenses:.2f}")
```

What the **for** loop does:

- Iterates through the **expenses** list (a sequence of tuples).
 - Prints each item in a formatted way.
 - Calculates the total while looping.
-

1. Function to Calculate Total Expenses

python

CopyEdit

```
def calculate_total_expenses(expenses):  
    """  
    Calculates total amount from a list of (category, amount) tuples.  
    Args:  
        expenses (list): List of tuples, each containing (category,  
amount)  
    Returns:  
        float: Total expenses  
    """  
    total = sum(amount for _, amount in expenses)  
    return total
```

2. Function to Display Expense Report

python

CopyEdit

```
def display_expense_report(expenses):  
    """  
    Prints a formatted list of expenses.  
    Args:  
        expenses (list): List of tuples, each containing (category,  
amount)  
    """
```

```
print("\n--- Expense Report ---")
for category, amount in expenses:
    print(f"{category}: ${amount:.2f}")
```

✓ 3. Function to Calculate Savings

python

CopyEdit

```
def calculate_savings(income, expenses_total):
    """
    Calculates remaining savings.
    Args:
        income (float): Monthly income
        expenses_total (float): Total of all expenses
    Returns:
        float: Savings (income - expenses)
    """
    return income - expenses_total
```

✓ Using These Functions Together

python

CopyEdit

```
# Sample data
monthly_income = 3000
expenses = [("Rent", 1200), ("Groceries", 400), ("Utilities", 200),
            ("Internet", 100)]

# Run functions
total = calculate_total_expenses(expenses)
display_expense_report(expenses)
savings = calculate_savings(monthly_income, total)

# Display result
print(f"\nTotal Expenses: ${total:.2f}")
print(f"Remaining Savings: ${savings:.2f}")
```

Goal

- Store a list of expenses.
 - Access individual expenses.
 - Use a loop to:
 - Print each expense.
 - Sum all expenses.
 - Apply changes (like a 10% reduction to simulate savings).
-

Example Code: List Manipulation and Iteration

python

CopyEdit

```
# List of weekly expenses in dollars
expenses = [50.0, 120.0, 35.5, 89.99, 60.0]

# Print each expense using a loop
print("Weekly Expenses:")
for expense in expenses:
    print(f"- ${expense}")

# Total expenses
total = 0
for expense in expenses:
    total += expense

print(f"\nTotal Weekly Expenses: ${total:.2f}")

# Apply a 10% budget cut to each expense
reduced_expenses = []
for expense in expenses:
    reduced = expense * 0.9 # 10% off
    reduced_expenses.append(reduced)
```

```
print("\nReduced Expenses After 10% Budget Cut:")
for reduced in reduced_expenses:
    print(f"- ${reduced:.2f}")
```

Concepts Covered

Concept	Example Used
List Creation	<code>expenses = [50.0, 120.0, ...]</code>
Accessing Elements	Inside the <code>for expense in expenses:</code> loop
For Loop Iteration	To print and calculate total
List Modification	New list <code>reduced_expenses</code> with 10% cut
List Appending	<code>reduced_expenses.append(reduced)</code>

Example: File Operations for a Budget Calculator

◆ Writing Expenses to a File

```
python
CopyEdit
expenses = [50.0, 120.0, 35.5, 89.99, 60.0]
filename = "expenses.txt"

# Write each expense to the file
with open(filename, "w") as file:
    for expense in expenses:
        file.write(f"{expense}\n")

print(f"Expenses written to {filename}.")
```

◆ Reading Expenses from a File

```
python
CopyEdit
```

```
# Read expenses from the file
read_expenses = []

with open(filename, "r") as file:
    for line in file:
        read_expenses.append(float(line.strip()))

print("Expenses read from file:")
for e in read_expenses:
    print(f"- ${e:.2f}")
```

What This Teaches

Feature	How It's Used
<code>with open(...)</code>	Automatically closes the file when done
<code>write()</code>	Saves each expense to a file as a new line
<code>read()</code> or loop over file	Reads the file line by line
<code>float(line.strip())</code>	Converts each line back into a number



Optional: Total Expenses After Reading from File

python

CopyEdit

```
total = sum(read_expenses)
print(f"\nTotal Expenses from File: ${total:.2f}")
```

Example: Exception Handling for File Operations in Budget Calculator

python

CopyEdit

```
filename = "expenses.txt"

try:
    # Attempt to open and read expenses from file
    with open(filename, "r") as file:
        lines = file.readlines()
        expenses = [float(line.strip()) for line in lines]
except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found.")
    expenses = []
except ValueError:
    print("Error: One or more lines in the file could not be converted to numbers.")
    expenses = []
else:
    # Runs only if no exception occurred
    print("Expenses loaded successfully:")
    for e in expenses:
        print(f"- ${e:.2f}")
    print(f"Total: ${sum(expenses):.2f}")
finally:
    # Always runs, even if exception occurred
    print("\nFile read attempt completed.")
```

What Each Clause Does

Clause	Purpose
<code>try</code>	Attempts to open the file and convert its contents to floats
<code>except</code>	Catches errors (missing file, bad data) and prevents program crash
<code>else</code>	Executes only if the <code>try</code> block was successful (no exceptions)
<code>finally</code>	Executes regardless of whether an exception occurred

Tip

You can use this pattern in many parts of the calculator:

- Reading/writing files
- Parsing user input (`float(input(...))`)
- Dividing numbers (handle `ZeroDivisionError`)